

Day 29: Visual SHAP — Bar, Beeswarm, Dependence, Waterfall

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 5, Day 2

Day 28 computed SHAP values and printed them as tables. No stakeholder reads a printed pandas Series in a report. Today turns the exact same underlying SHAP values into the four visuals practitioners actually build and share: a global bar chart, a beeswarm plot, dependence plots, and a waterfall breakdown — each covered with what it is, why it works, a real-world scenario, the implementation, a line-by-line breakdown, and when to reach for it versus when not to.

1. The SHAP Explanation Object

Topic specification: Day 28 called `explainer.shap_values(X)`, which returns a plain numpy array of numbers. Today calls `explainer(X)` instead, which returns a `shap.Explanation` object bundling the SHAP values, the base value, and the original feature values together in one container.

Explanation: Every function in the modern `shap.plots` module (`bar`, `beeswarm`, `scatter`, `waterfall`) expects an `Explanation` object, not a raw array — the plotting functions need the original feature values (to color dots by high/low, for example), not just the SHAP contributions alone.

Real-world scenario: A data science team migrating an older notebook built on `shap_values()` arrays finds that none of the newer, better-documented `shap.plots` functions work directly on that array — switching to `explainer(X)` and its `Explanation` object unlocks the entire modern plotting API without recomputing anything.

Implementation:

```
explainer = shap.TreeExplainer(classifier)
explanation = explainer(X_transformed)
explanation.feature_names = list(feature_names)
```

Line-by-line explanation

- `explainer(X_transformed)` — calling the explainer object directly (its `__call__` method) instead of `.shap_values()`; same underlying computation, different, richer return type.
- `explanation.feature_names = list(feature_names)` — attaches human-readable column names to the `Explanation` object, since the plotting functions display these directly on the chart's axis labels.
- No SHAP values are recomputed between Day 28 and Day 29 — this is purely a different way of packaging the same numbers for a different consumer (a plotting function instead of a printed table).

Why choose this? The `Explanation` object is required for every modern `shap.plots` function, carries feature names and raw feature values alongside the SHAP values, and is the API shap's own documentation and examples are written against going forward.

Why NOT this (tradeoffs)? For a quick one-off numeric check (like Day 28's manual base value + contributions sum), the plain array from `shap_values()` is simpler and has less object overhead — the Explanation object's extra structure only pays off once actual plotting is involved.

2. Global Bar Plot

Topic specification: `shap.plots.bar(explanation)` draws mean |SHAP value| per feature as a horizontal bar chart — the direct visual counterpart to the numeric table from Day 28's global SHAP importance comparison.

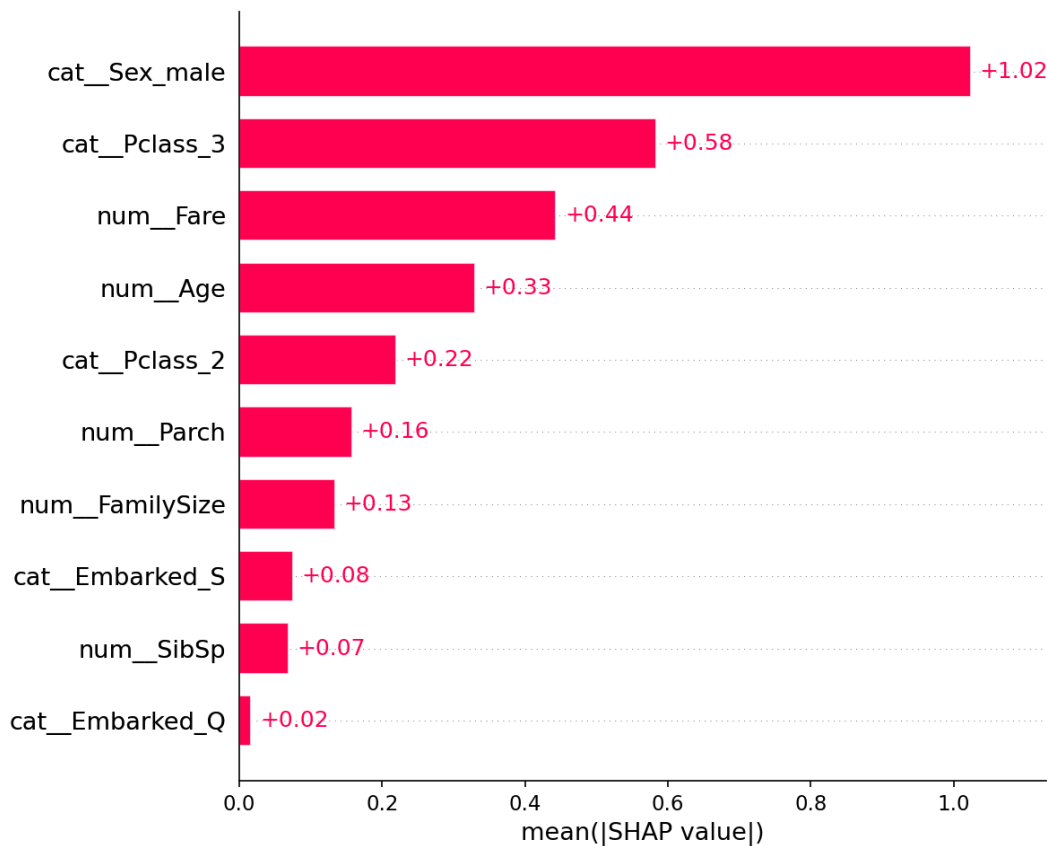
Explanation: Internally this is the same aggregation as Day 28's Topic 5 (average the absolute SHAP value across every row, per feature) — the only difference is the output is a matplotlib figure instead of a printed pandas Series.

Real-world scenario: A model-governance slide deck needs one clean, self-explanatory chart showing which features drive a credit-risk model overall, without requiring the audience to read a table of numbers — the bar plot is the simplest, least ambiguous way to present that ranking visually.

Implementation:

```
plt.figure()
shap.plots.bar(explanation, show=False)
plt.tight_layout()
plt.savefig("shap_bar_global.png", dpi=150)
plt.close()
```

Output:



Actual output: [shap_bar_global.png](#), generated from this lesson's fitted model.

Line-by-line explanation

- `plt.figure()` — starts a fresh matplotlib figure so this plot doesn't draw on top of a previous one still open in memory.
- `show=False` — tells shap not to call `plt.show()`, which would try to open an interactive window; unnecessary and unsupported in a headless/scripted environment.
- `plt.savefig(..., dpi=150)` — writes the figure to a PNG file at a resolution sharp enough for a slide or report, before the figure is discarded.
- `plt.close()` — frees the figure from memory; without this, generating many plots in one script run will slowly leak memory.

Why choose this? It's the simplest, fastest visual to generate and read — a single ranked bar chart with no risk of misinterpretation, well suited for an executive summary or a governance review.

Why NOT this (tradeoffs)? It collapses each feature down to one magnitude number, exactly like gain-based importance did — it shows nothing about direction (does high Fare push survival up or down?) or how SHAP values vary row to row, which the beeswarm plot addresses directly.

3. Beeswarm Plot

Topic specification: `shap.plots.beeswarm(explanation)` plots one dot per row per feature: the dot's horizontal position is that row's SHAP value, and the dot's color encodes whether that

row's actual feature value was high or low.

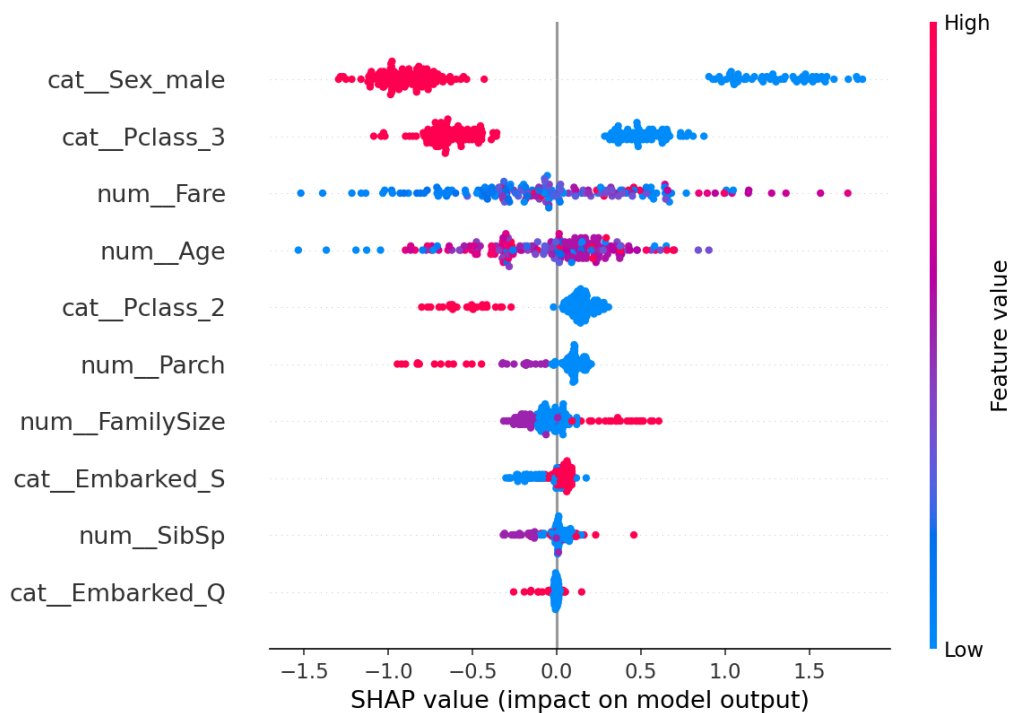
Explanation: Because every individual row appears as its own dot, the beeswarm plot shows both the overall magnitude of a feature's effect (dots spread wide vs clustered near zero) and its direction (do high-value dots cluster on the positive or negative side) in one image — information a single bar or number cannot carry.

Real-world scenario: A hospital readmission model's beeswarm plot shows that high values of one lab result push risk up (red dots on the positive side) while high values of another push risk down (red dots on the negative side) — a clinician can read both the importance ranking and the clinical direction of each factor from one chart.

Implementation:

```
plt.figure()
shap.plots.beeswarm(explanation, show=False)
plt.tight_layout()
plt.savefig("shap_beeswarm.png", dpi=150)
plt.close()
```

Output:



Actual output: shap_beeswarm.png, generated from this lesson's fitted model.

Line-by-line explanation

- `shap.plots.beeswarm(explanation)` — automatically sorts features top-to-bottom by importance (same ordering as the bar plot) and plots every row's SHAP value as a dot on each feature's row.
- Dot color comes directly from the Explanation object's stored feature values — this is exactly why Topic 1's Explanation object (which carries feature values, not just SHAP values) is required here; a plain

shap_values() array has nothing to color the dots with.

- Dots that overlap horizontally are jittered vertically (hence 'beeswarm') so density is visible instead of dots stacking invisibly on top of each other.

Why choose this? It's the single richest global SHAP visualization — showing importance ranking, the spread of effect sizes, and direction simultaneously, replacing both the bar plot and a mean |SHAP| table in one image.

Why NOT this (tradeoffs)? With very large datasets or many features, a beeswarm plot can become visually dense and harder to read than a simple bar chart — for a quick executive summary where direction doesn't need to be shown, the bar plot from Topic 2 communicates faster.

4. Dependence / Scatter Plot

Topic specification: `shap.plots.scatter(explanation[:, feature_name])` plots one feature's actual value on the x-axis against that same feature's SHAP contribution on the y-axis, for every row — revealing the shape of the relationship, not just its average strength.

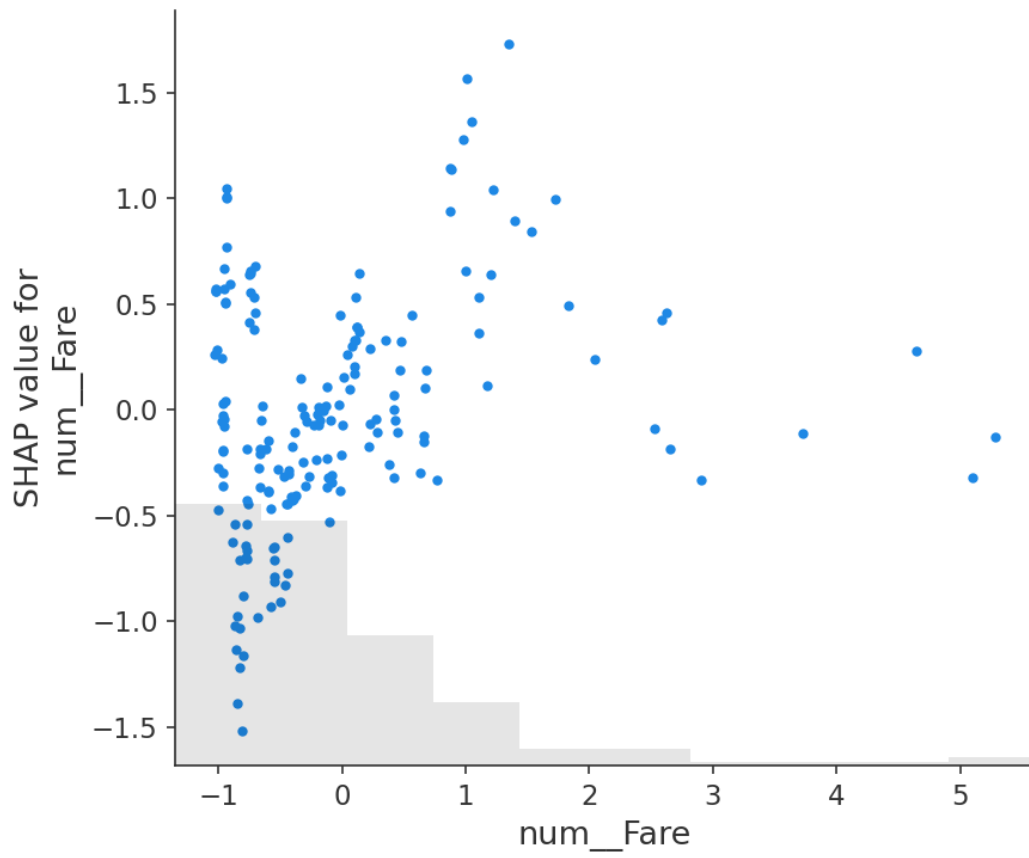
Explanation: A feature's effect on a prediction isn't always linear. A dependence plot can reveal a threshold effect (no impact until a certain value, then a sharp jump), a plateau (impact flattens out past a point), or a genuinely linear relationship — details a single importance number cannot distinguish between.

Real-world scenario: An e-commerce pricing model's dependence plot on 'days since last purchase' shows the SHAP contribution barely changes for 0-30 days, then drops sharply past 30 days — revealing a meaningful business threshold (a customer becoming 'at risk of churn') that a single average importance score would have completely hidden.

Implementation:

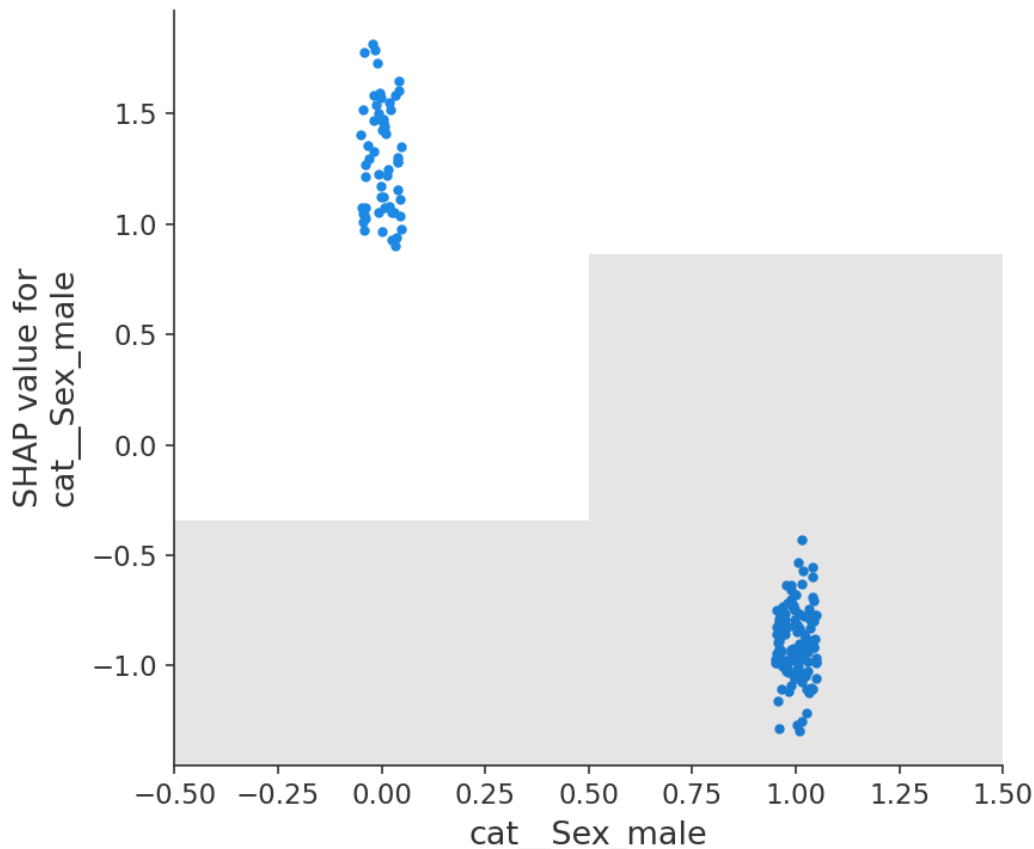
```
plt.figure()
shap.plots.scatter(explanation[:, "num__Fare"], show=False)
plt.tight_layout()
plt.savefig("shap_dependence_num_Fare.png", dpi=150)
plt.close()
```

Output:



Actual output: [shap_dependence_num_Fare.png](#) — Fare's dependence plot.

Output:



Actual output: [shap_dependence_cat_Sex_male.png](#) — Sex's dependence plot.

Line-by-line explanation

- `explanation[:, "num__Fare"]` — indexes the Explanation object down to just one feature's column across every row, using the feature name attached in Topic 1.
- `shap.plots.scatter(...)` — plots that one feature's actual value against its SHAP contribution; by default, shap automatically picks a second feature to color the dots by, surfacing potential interaction effects between the two.
- Repeating this call per feature (Fare, then Sex, in this lesson) produces one focused plot per feature, rather than trying to cram every feature's relationship into a single chart.

Why choose this? It's the only plot in this lesson that reveals the shape of a feature's effect — linear, threshold, or plateau — which matters directly for decisions like where to draw a business rule cutoff, not just which feature to pay attention to.

Why NOT this (tradeoffs)? It only shows one feature (or one feature pair) at a time — reviewing every feature this way for a model with dozens of columns is slow. It's best used selectively, on the top few features the bar or beeswarm plot already flagged as important.

5. Waterfall Plot

Topic specification: `shap.plots.waterfall(explanation[row_index])` draws Day 28's single-prediction breakdown — base value plus every feature's signed contribution equals the final

prediction — as a literal step-by-step waterfall chart instead of a printed table.

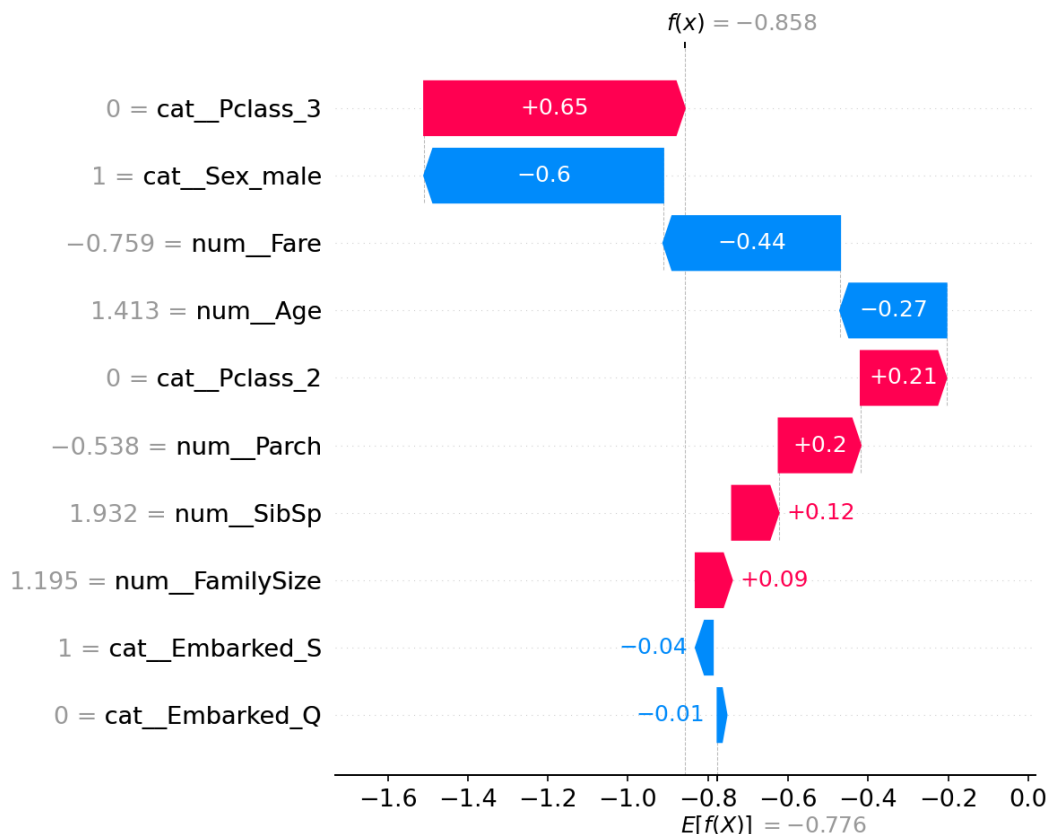
Explanation: The chart starts at the base value on one side, then draws each feature's contribution as a step up or down, in order of magnitude, landing exactly on the row's final predicted value — visually preserving the same additive guarantee verified numerically in Day 28.

Real-world scenario: An insurance claims adjuster disputing a customer's risk score pulls up the waterfall plot for that customer's specific prediction and can point to the exact bars — 'your claim history added this much risk, your vehicle age reduced it by this much' — instead of reading off a table of numbers the customer would need help interpreting.

Implementation:

```
plt.figure()
shap.plots.waterfall(explanation[0], show=False)
plt.tight_layout()
plt.savefig("shap_waterfall_row0.png", dpi=150)
plt.close()
```

Output:



Actual output: shap_waterfall_row0.png — one passenger's full breakdown.

Line-by-line explanation

- `explanation[0]` — indexes the Explanation object down to a single row (test row 0), the same row Day 28 explained numerically.

- `shap.plots.waterfall(...)` — automatically sorts that row's feature contributions by magnitude and draws them as sequential steps from the base value to the final prediction.
- Features with small contributions are automatically grouped into a single 'other features' bar by default, keeping the chart readable even when there are many columns.

Why choose this? It makes an individual prediction fully auditable and explainable to a non-technical audience in one glance — exactly the kind of artifact a regulated decision (credit, insurance, hiring) needs to be defensible.

Why NOT this (tradeoffs)? It only explains one row at a time, the same limitation as Day 28's Topic 4 — for understanding overall model behavior across many predictions, the bar or beeswarm plot is the right tool, not a waterfall repeated hundreds of times.

Interview Preparation — Technical Questions

Q: What's the practical difference between `explainer.shap_values(X)` and `explainer(X)`?

A: `shap_values(X)` returns a plain numpy array of SHAP values only. `explainer(X)` returns a `shap.Explanation` object that additionally bundles the base value and the original feature values — required input for every function in the modern `shap.plots` module.

Q: Why does the beeswarm plot need the original feature values, not just the SHAP values?

A: Each dot's color in a beeswarm plot encodes whether that row's actual feature value was high or low. A plain `shap_values()` array only contains the contributions, not the original values, so it has nothing to color the dots with — this is why the `Explanation` object is required.

Q: What does a bar plot show that a beeswarm plot doesn't, and vice versa?

A: A bar plot shows only the aggregate magnitude (mean |SHAP value|) per feature — one number per feature. A beeswarm plot shows every individual row's contribution and its direction, revealing the full distribution and whether a feature's effect is consistent or highly variable across rows — information a single bar cannot carry.

Q: Why might a dependence plot reveal a threshold effect that a global importance ranking completely misses?

A: Global importance summarizes a feature's average effect into one number, discarding information about how that effect varies across the feature's range. A dependence plot shows every row's actual value against its contribution, making a non-linear pattern — like no effect below a threshold and a sharp jump above it — visible, when the average alone would look like a moderate, uniform effect.

Q: What guarantees does a waterfall plot visually preserve from Day 28's SHAP additivity?

A: The steps in a waterfall plot — starting at the base value and adding or subtracting each feature's contribution — always land exactly on the model's actual predicted value for that row, the same additive guarantee verified numerically in Day 28 by manually summing base value and contributions.

Q: Why is `plt.close()` necessary after saving each SHAP plot in a script that generates several?

A: Matplotlib keeps figures in memory until explicitly closed. Without `plt.close()` after each `plt.savefig()` call, every subsequent plot accumulates in memory alongside the previous ones, risking increasing memory use and, in some cases, plots overlapping or bleeding into each other.

Q: Why use `matplotlib.use('Agg')` at the top of a script that generates SHAP plots?

A: 'Agg' is a non-interactive, headless-safe matplotlib backend that renders directly to image files without requiring a display — necessary in scripted, server-side, or sandboxed environments where no graphical window can be opened.

Q: In `shap.plots.scatter`, what does the automatic color-by-second-feature behavior reveal?

A: By default, `shap.plots.scatter` colors each dot by the value of a second feature it detects has the strongest interaction with the one being plotted, surfacing potential interaction effects — for example, whether a feature's impact differs depending on another feature's value — beyond what a single-feature dependence view alone would show.

Interview Preparation — Theoretical Questions

Q: Why is visualizing SHAP values often more valuable in practice than printing them, even though the underlying information is identical?

A: Stakeholders — executives, regulators, customers, clinicians — process visual patterns (clusters, trends, magnitude comparisons) far faster and more reliably than scanning a table of decimal numbers. The information content is the same, but a well-chosen visualization dramatically lowers the cognitive cost of extracting the right conclusion from it, which is why the same values from Day 28 are worth re-expressing visually here.

Q: Why can a beeswarm plot reveal that a feature is important but inconsistent, in a way a bar chart alone cannot?

A: A bar chart reduces a feature's contributions to their mean absolute value — a single number. A beeswarm plot preserves the full spread of individual contributions as visible dots; a feature whose dots are widely scattered on both sides of zero has a real but inconsistent effect (helping some rows, hurting others), which averages to a moderate bar height that looks identical to a feature with a small, consistent effect — the beeswarm plot is what distinguishes the two cases.

Q: Why might a dependence plot's shape matter more for a business decision than the feature's overall importance rank?

A: A business decision often needs an actionable cutoff or threshold, not just a ranking of which factors matter. A feature ranked fifth in importance but showing a sharp threshold effect in its dependence plot can directly inform a concrete policy (e.g., 'flag accounts past this value'), while the top-ranked feature showing a smooth, gradual relationship may offer no clean actionable cutoff at all.

Q: Why is a waterfall plot considered more defensible than a plain probability score when explaining a decision to an affected individual?

A: A probability score alone states an outcome with no reasoning attached. A waterfall plot traces the exact, additive path from a neutral starting point (the base value) through each specific factor's contribution to that individual's outcome, giving a concrete, falsifiable account of why the decision came out the way it did — the kind of reasoning regulated decisions are increasingly required to provide.

Q: Why does converting SHAP values into an Explanation object rather than working with raw arrays reflect a broader pattern in tooling design?

A: Raw arrays carry numbers but strip away the context (feature names, original values, base value) needed to interpret them correctly. Wrapping related data into one structured object reduces the chance of a bug where numbers and their meaning drift out of sync (e.g. a SHAP value array misaligned with the wrong feature name list) — the same principle behind using a Pipeline object (Day 21) instead of loose preprocessing steps.

Q: Why might relying only on a global visualization (bar or beeswarm) still miss the same individual-fairness concerns that per-row analysis (Day 28's Topic 4, or a waterfall plot) catches?

A: Global visualizations aggregate across every row, which can average out a problem that affects a small but meaningful subgroup — a feature could show a moderate, balanced-looking average effect globally while systematically disadvantaging one specific segment whenever it occurs. Per-row

explanations and targeted subgroup analysis are needed to catch that pattern; the global view alone cannot.

Q: In what sense does a dependence plot's automatic interaction-coloring connect back to the additivity assumption underlying SHAP values?

A: Base SHAP values attribute each feature's contribution independently and additively. An interaction between two features means one feature's effect genuinely depends on another's value — the dependence plot's coloring is a way of visually surfacing where that assumption of clean, independent additivity is masking a real joint effect between two features, without abandoning the additive framework itself.

Q: Why does producing PNG files rather than only interactive plots matter for how this work gets used downstream?

A: Interactive plots require a live Python environment to view. Saved PNG files can be embedded directly into a slide deck, a PDF report, a dashboard, or an email — the artifact becomes portable and consumable by anyone, including people with no Python installed at all, which is usually the actual audience for a model explanation.

Key Takeaway and What's Next

Key takeaway: Day 28 established that SHAP values are correct and additive; Day 29 established that the same values, visualized correctly, are what actually gets shared with anyone who isn't reading Python output directly — a bar plot for a quick global summary, a beeswarm plot for the full global picture with direction, a dependence plot for one feature's shape, and a waterfall plot for one individual's full explanation.

What typically follows visual interpretability

- Partial dependence and ICE plots — a model-level (not SHAP-specific) view of how a prediction changes as one feature varies, useful as a cross-check against SHAP's dependence plots.
- Smarter hyperparameter search — RandomizedSearchCV and Bayesian/Optuna-style optimization, once a model's behavior is understood well enough to be worth tuning further.
- Model persistence and serving — saving the fitted pipeline and its explainer together with joblib, so explanations can be generated for new predictions in production, not just during analysis.